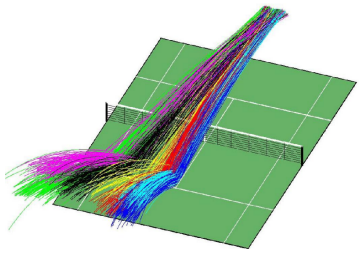
Scientific Programming
Chapter 4 - Differential Equations and Integration

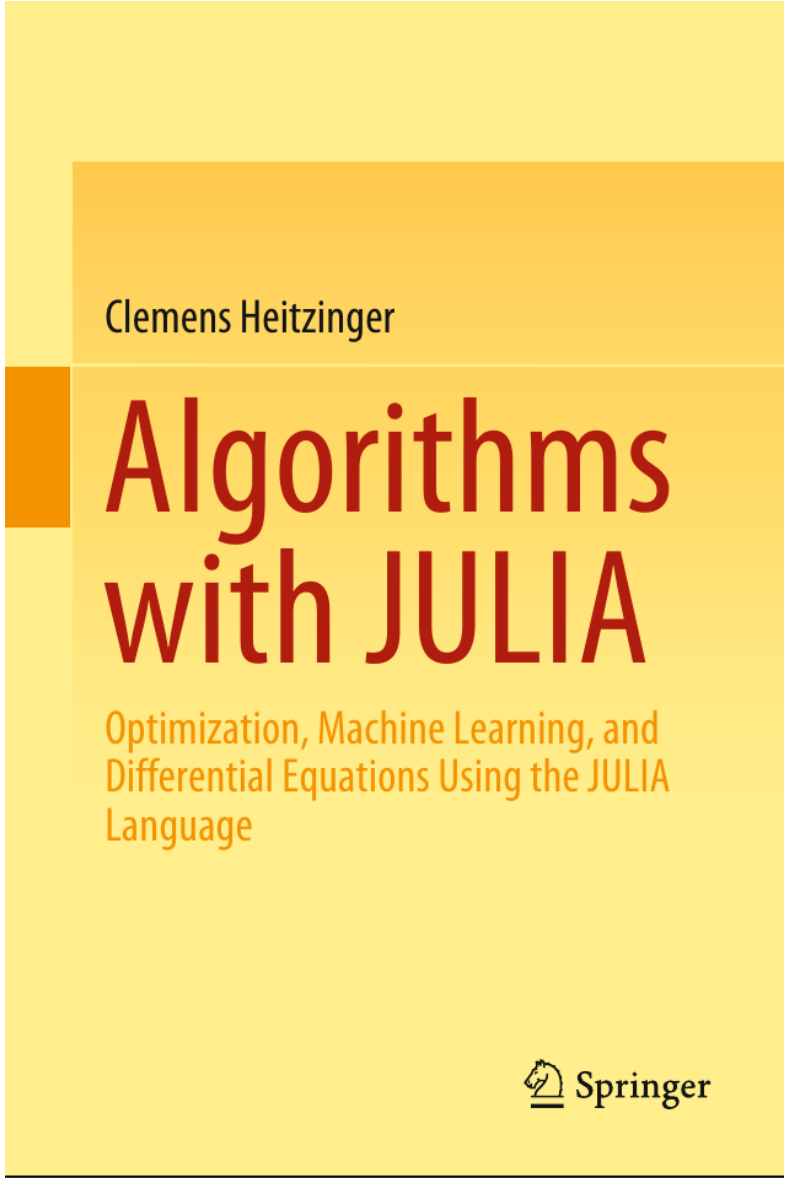
Jean-Sébastien Lerat



HEH be
Sciences
et technologies

2023-2024

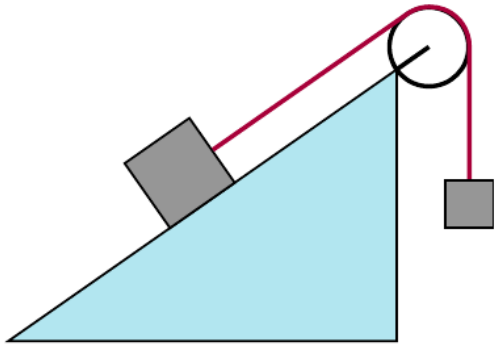
Main Reference

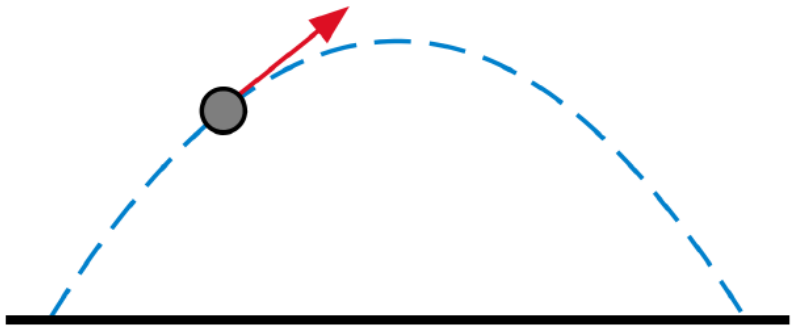


Introduction

Ordinary Differential Equation (ODE)

are equations containing derivatives of one or more functions depending on a single on a single variable (time).





ODE are frequently used as mathematical models.

Partial Differential Equations (PDE)

are equations containing derivatives of one or more functions depending on a single on a several independent variables (time, space, . . .).

Introduction

When there is no analytical solution
$\Rightarrow$ **approximate solution using numerical methods**

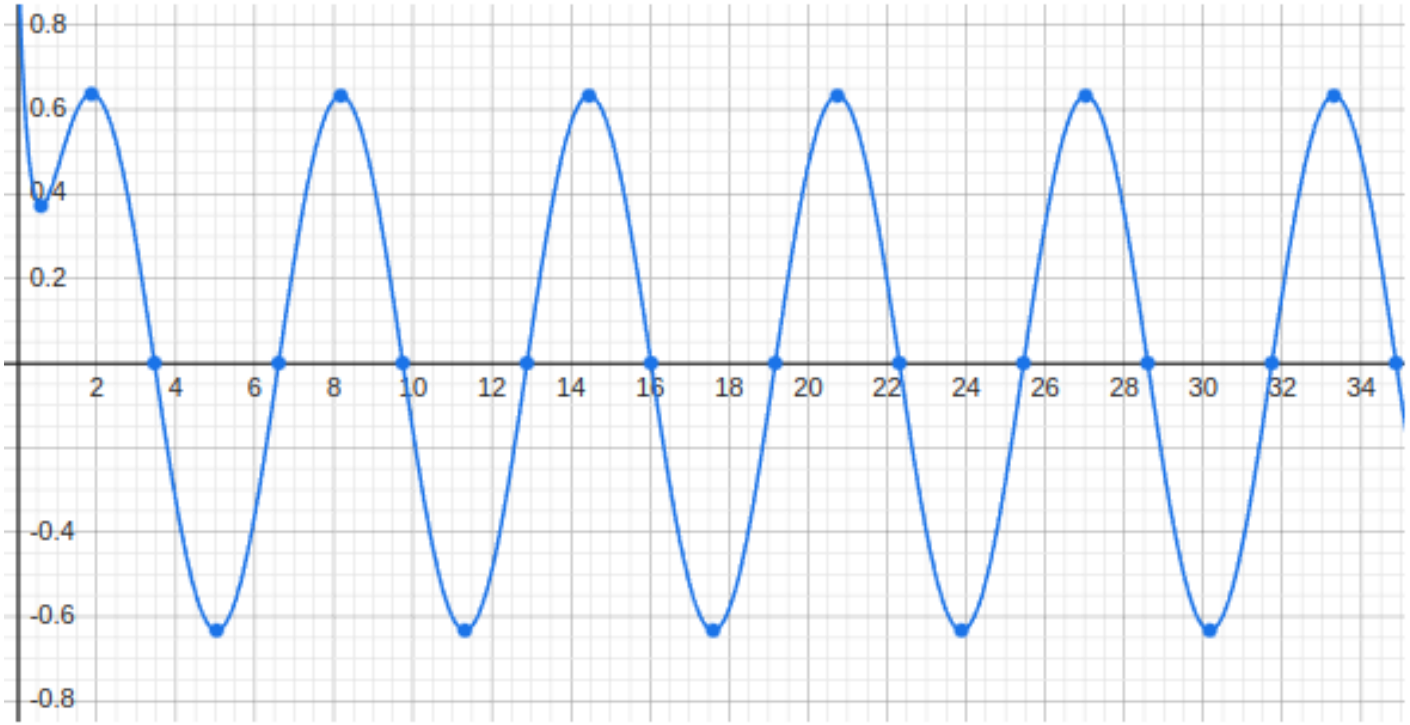
ODE version of a Cauchy problem
(*initial value problem* or *boundary value problem*)

Given an interval $[a, b] \subset \mathbb{R}$, $y_0 \in \mathbb{R}$ and $f(t, u) : [a, b] \times \mathbb{R} \to \mathbb{R}$, we want to define $u(t)$ such that

$$\begin{cases} u'(t) &= f\left(t, u(t)\right)), \qquad t \in [a, b] \\ u(t_0) &= u_0 \end{cases}$$

Introduction

Original (unknown) $\mathbf{u(t)} = \dfrac{\mathbf{1}}{\mathbf{5}}\left(\mathbf{6e^{-3t}} + \mathbf{3}\,\sin(\mathbf{t}) - \cos(\mathbf{t})\right)$



But we have a mathematical model:

$$\begin{cases} u'(t) &= 2\,\sin(t) - 3\,u(t), \qquad t \in [0, 20] \\ u(0) &= 1 \end{cases}$$

Introduction

Some mathematical requirements for **existence** and **unicity**

(Continuity and Lipschitz Continuity)

Content

1. Introduction
2. Approximate ODE
   - Principle
   - Euler method
   - Runge-Kutta methods
   - Example of RK2 First-Order
   - Vector-Valued Function
   - Higher Order
3. Numerical Integration
   - Principle
   - Lagrange Polynomial
   - Newton–Cotes rule
   - Newton–Cotes Formulas
4. Exercises
   - Hawk-Eye for Tennis
   - Integration

Approximate ODE

Principle

Reminder (Ch. 2): Taylor series of order n

is a mathematical tool to approximate a function $f(\bullet)$ in the neighbourhood of a, f is at least n times derivable:

$$f(x) \approx f(a) + \frac{(x-a)^1}{1!} f'(a) + \frac{(x-a)^2}{2!} f''(a) + \ldots + \frac{(x-a)^n}{n!} f^n(a) + \varepsilon$$

When $x \to a$, the error is $\varepsilon = 0$

And evaluate the Taylor serie:
n **points equally distributed over the interval** $[a, b]$

The step is $h = \dfrac{b-a}{n-1}$ and t at step $i \in [0, n-1]$ is $t_i = a + ih$

Approximate ODE

Euler method

The Euler method

is to use a Taylor serie of order $n = 1$:

$$f(x) \approx f(a) + \frac{x-a}{1!} f'(a) + \varepsilon$$

ε is the error (quantify later).

Approximate t_i in the neighboourdhoud of $a = t_{i-1}$ becomes:

$$u(t_i) \approx u(t_{i-1}) + (t_i - t_{i-1}) u'(t_{i-1})$$

Simplification with Cauchy statement $\Rightarrow u'(t_{i-1}) = f\left(t_{i-1}, u(t_{i-1})\right)$
and difference between steps $(t_i - t_{i-1})$ is h:

$$u(t_i) \approx u(t_{i-1}) + h\, f\left(t_{i-1}, u(t_{i-1})\right)$$

Approximate ODE

Euler method

Previous example

$$\begin{cases} u'(t) &= 2\,\sin(t) - 3\,u(t), \qquad t \in [0, 20] \\ u(0) &= 1 \end{cases}$$

Evaluate $n = 100 \qquad \Rightarrow h = \dfrac{20-0}{100-1} = \qquad \Rightarrow t_i = 0.2020\ i$

$$\begin{aligned}
u(t_0 = 0) &= 0 \\
u(t_1 = h = 0.2020) &= u(t_0 = 0) + h\ f\ (t_0 = 0, u(t_0 = 0)) \\
&= 1 + 0.2020\ [2\,\sin(t_0 = 0) - 3\ u(t_0)] \qquad \text{Cauchy} \\
&= 1 + 0.2020\ [2 \times 0 - 3 \times 1] \\
&= 0.3940 \\
u(t_2 = 2h = 0.4040) &= u(t_1 = h) + h\ f\ (t_1 = h, u(t_1) = 0) \\
&= 0.3940 + 0.2020\ [2 \times 0.2006 - 3 \times 0.3940] \\
&= 0.2362784 \\
u(t_3) &= \ldots
\end{aligned}$$

Approximate ODE

Euler method – Error

By Taylor serie (order 1, ε), after an iteration, the error is

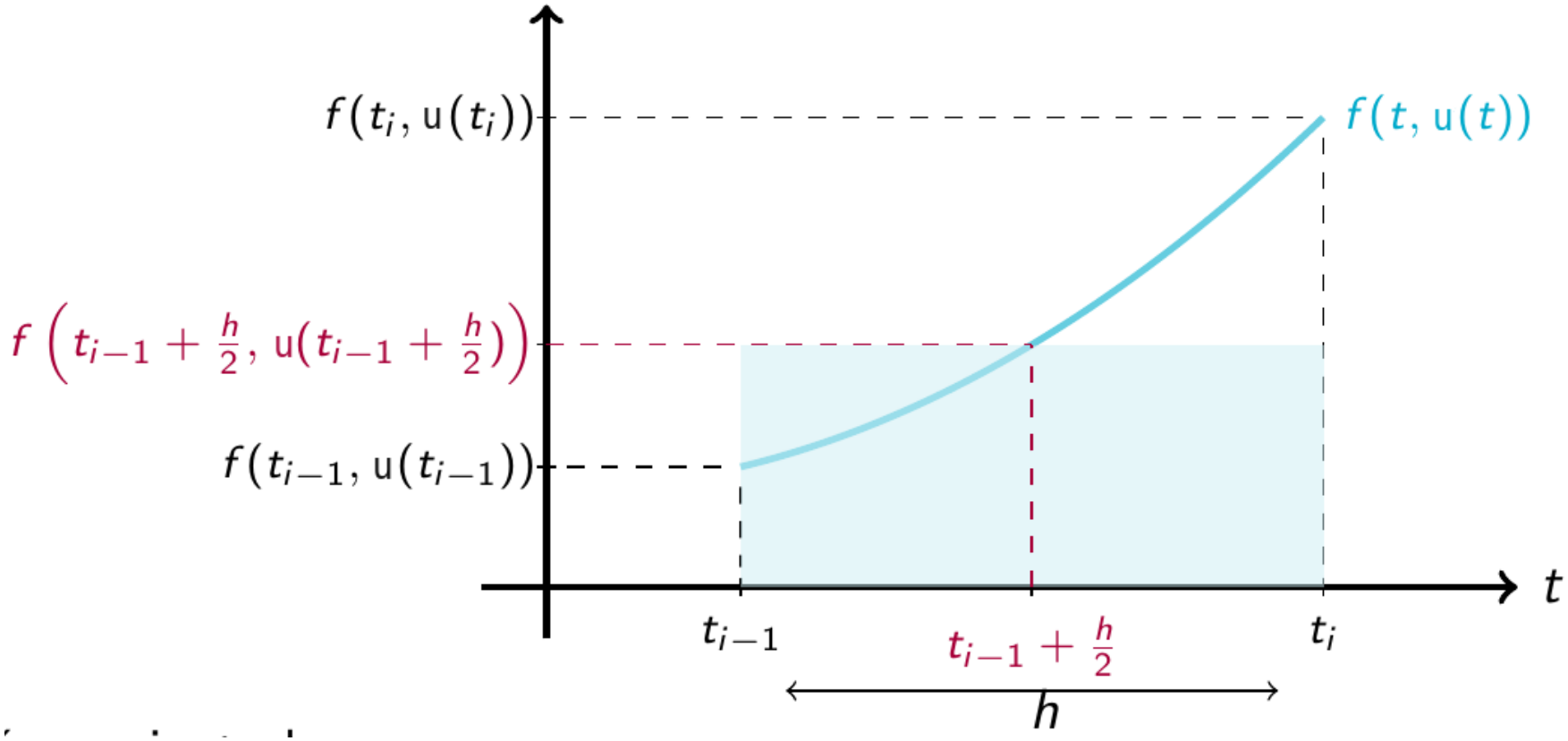
$$\underbrace{\overbrace{u(t_1)}^{Approximation} - \underbrace{u_1}_{\text{real value}}}_{\tilde{e}_i} \leq M\ h^2 \qquad \text{where M is a constant}$$

The global error (Lipschitz) is also bounded by $\leq Kh$
where K is a constant that does not depend of h
h depends of $n \Rightarrow$ does not depend of n

Approximate ODE

Runge-Kutta methods

Euler method is $u(t_i) = u(t_{i-1} + h\phi(t_{i-1}, u(t_{i-1}), h)$
where $\phi(t, h, u(t)) = f(t, u(t))$



Better approximation with inner steps (subdivide h) $\Rightarrow$ redefine ϕ

Approximate ODE

Runge-Kutta methods

Runge-Kutta with two stages uses intermediate values (k_1, k_2)

$$\phi(t, h, u(t)) = k_2$$

where

$$\begin{aligned} k_1 &= f(t_{i-1}, u(t_{i-1})) \\ k_2 &= f\left(t_{i-1} + \frac{h}{2}, u(t_{i-1}) + \frac{h}{2}k_1\right) \end{aligned}$$

is like Euler method with more points.

Approximate ODE

Runge-Kutta methods

But Runge-Kutta methods aims to find $f(t, u(t))$ with area (=integrals):

$$\begin{aligned} u'(t) &= f(t, u(t)) \\ \int_{t_{i-1}}^{t_i} u'(t) &= \int_{t_{i-1}}^{t_i} f(t, u(t))\mathrm{d}t \\ u(t_i) - u(t_{i-1}) &= \int_{t_{i-1}}^{t_i} f(t, u(t))\mathrm{d}t \\ u(t_i) &= u(t_{i-1}) + \int_{t_{i-1}}^{t_i} f(t, u(t))\mathrm{d}t \end{aligned}$$

It is possible to demonstrate with the two-variable Taylor series development:

$$u(t_i) = u(t_{i-1}) + h(\overbrace{b_1 k_1 + b_2 k_2}^{\phi})$$

with $k_1 = f(t_{i-1}, u(t_{i-1}))$ $\qquad k_2 = f(t_{i-1} + c_2\ h, a_{2,1} k_1\ u(t_{i-1}))$

Approximate ODE

Runge-Kutta methods

General form of Runge-Kutta with s stages:

$$u(t_{i+1}) = u(t_i) + h \sum_{j=1}^{s} b_j \; k_j$$

where $k_j = f\left(t_i + c_j \; h, u(t_i) \; + h \sum_{n=1}^{j-1} a_{j,n} \; k_n\right)$

There are conditions to determine $a_{j,n}$, c_j and b_j.
Without mathematical details, lets use the **Runge–Kutta matrix** (also called *Butcher tableau*)

Pros: like a higher order of Talor series $\Rightarrow$ error is lower

Approximate ODE

Runge-Kutta methods

Runge–Kutta matrix

$$
\begin{array}{c|ccccc}
0 & & & & & \\
c_2 & a_{2,1} & & & & \\
c_3 & a_{3,1} & a_{2,2} & & & \\
\vdots & \vdots & & \ddots & & \\
c_s & a_{s,1} & a_{s,2} & \dots & a_{s,s-1} & \\
\hline
 & b_1 & b_2 & \dots & b_{s-1} & b_s
\end{array}
$$

defines one Runge-Kutta method

Approximate ODE

Runge-Kutta methods

$$\begin{array}{c|cccc} 0 & & & & \\ \frac{1}{2} & \frac{1}{2} & & & \\ \frac{1}{2} & 0 & \frac{1}{2} & & \\ 1 & 0 & 0 & 1 & \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array}$$

Figure: Classical RK4

$$\begin{array}{c|cccc} 0 & & & & \\ \frac{1}{3} & \frac{1}{3} & & & \\ \frac{2}{3} & -\frac{1}{3} & 1 & & \\ 1 & 1 & -1 & 1 & \\ \hline & \frac{1}{8} & \frac{3}{8} & \frac{3}{8} & \frac{1}{8} \end{array}$$

Figure: RK4 variant (3/8-rule)

$$\begin{array}{c|c} 0 & \\ \hline & 1 \end{array}$$

Figure: RK1 (Euler method)

$$\begin{array}{c|cc} 0 & & \\ \frac{1}{2} & \frac{1}{2} & \\ \hline & 0 & 1 \end{array}$$

Figure: RK2 (midpoint method)

Approximate ODE

Example of RK2 First-Order

Carbon-14 dating (^{14}C) is to determine the ratio between ^{14}C and total of carbon ($N(t)$):

$$\frac{^{14}C}{C_{\text{total}}}$$

The decay constant of ^{14}C is

$$\lambda = \frac{\ln(2)}{t_{1/2}} \qquad \text{, where } t_{1/2} = 5.74 \times 10^3 \text{ years is the half-life of } ^{14}C$$

$n(t)$ denotes the amount of ^{14}C present at time t, then

$$n'(t) = -\lambda\ n$$

Analytic solution: $n(t_0)\ e^{-\lambda\ t}$
but let's say you cannot find an analitic solution.

Approximate ODE

Example of RK2 First-Order

In the year of grace 1988, on March 25, the best day ever Your found an oak statue of the god Belegkarnil.

The mesured proportion of ^{14}C is

$$n_{1988} = \frac{^{14}C}{C_{\text{total}}} = \frac{1}{80^{12}}$$

Based on the actual alive wood:

$$n_0 = \frac{^{14}C}{C_{\text{total}}} = \frac{1}{10^{12}}$$

How old?

1. Convert $t_{1/2}$ in standard unit:
   $5.74 \times 10^3 \times (24 \times 60 \times 60) \times 365.25 = 1.81140624 \times 10^{11}$ seconds
2. Compute $\lambda = \dfrac{\ln(2)}{1.81140624 \times 10^{11}} = 3.826569464395492 \times 10^{-12}$
3. Formulate Cauchy, solve with RK to find how much seconds Δt between n_0 and n_{1988}

Approximate ODE

Example of RK2 First-Order

RK2

$$
\begin{aligned}
k_1 &= f(t_i + c_1\ h, u(t_i) + h \sum_{n=1}^{1-1} a_{1,n}\ k_n) \\
&= f(t_i\ h, u(t_i)) \\
k_2 &= f(t_i + c_2\ h, u(t_i) + h \sum_{n=1}^{2-1} a_{2,n}\ k_n) \\
&= f(t_i + \frac{1}{2} h, u(t_i) + h\ a_{2,1}\ k_1) \\
&= f(t_i + \frac{1}{2} h, u(t_i) + \frac{1}{2} h\ k_1) \\
u(t_{i+1}) &= u(t_i) + h\,(b_1\ k_1 + b_2\ k_2) \\
&= h\,(0 \times k_1 + 1 \times k_2) \\
&= h\ k_2
\end{aligned}
$$

Cauchy

$$
\begin{cases}
n'(t) &= -3.8266 \times 10^{-12}\ n(t) \\
n(t_0) &= \dfrac{1}{10^{12}}
\end{cases}
$$

Approximate ODE

Vector-Valued Function

From

$$u : \mathbb{R} \mapsto \mathbb{R} : t \mapsto \vec{u}(t)$$

To

$$\vec{u} : \mathbb{R} \mapsto \mathbb{R}^n : t \mapsto \vec{u}(t) = \big(u_1(t), u_2(t), \ldots, u_n(t)\big)$$

Let's change the notation (coordinates and iterations are usually subscripted)
Iteration becomes superscript and coordinates becomes subscript.

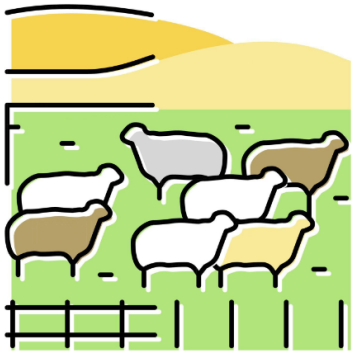
Cauchy problem ($f : \mathbb{R}^{n+1} \mapsto \mathbb{R}^n$) becomes:

$$\left\{ \begin{array}{rcl} u'(t) & = & f\big(t, \vec{u}(t)\big) = f\big(t, u_1(t), u_2(t), \ldots, u_n(t)\big) \in \mathbb{R}^n \\ \vec{u}(t^0) & = & \vec{u^0} \in \mathbb{R}^n \end{array} \right.$$

Approximate ODE

Vector-Valued Function

Example: a pasture



Proportion of animals: $E \geq 0$

Grass (g)

Growth rate: $R_g > 0$

Weed[a] (w)

Growth rate: $R_w > 0$

[a] French: mauvaise herbe

Approximate ODE

Vector-Valued Function

RK2 for first-order EDO in $\mathbb{R}^2$

$$\begin{cases} \partial_t g(t) &= R_g g(t)(1 - g(t) - 0.6wC) \\ \partial_t w(t) &= R_w w(t)(1 - w(t) - 1.07gC^{-1}) \\ g(t_0) &= g_0 \\ w(t_0) &= w_0 \end{cases}$$

where $C = \begin{cases} 1 & \text{if } E = 0 \text{ and } g(t) = 0 \\ \dfrac{E + g(t)}{0.31E + g(t)} & \text{Otherwise} \end{cases}$

Note: $g'(t)$ was a simple notation for $\partial_t g(t)$

What happens when $R_g = 0.27$ and $R_w = 0.4$ for different value of E?

$E = 0$ (no animals) $\quad E = 0.1 \quad E = 0.3 \quad E = 0.4$

$\Rightarrow \forall (g_0, w_0)$, does the system always converge to the same value (g^*, w^*)?

Approximate ODE

Vector-Valued Function

Cauchy

$$\begin{cases} \vec{u}'(t) & = & \Big(0.27g(t)(1-g(t)-0.6wC), 0.4w(t)(1-w(t)-1.07gC^{-1})\Big) \\ \vec{u}(t_0) & = & \Big(g_0, w_0\Big) \end{cases}$$

Approximate ODE

Vector-Valued Function

RK2

$$
\begin{aligned}
\vec{k}_1 &= f(t^i + c_1\ h, \vec{u}(t^i) + h \sum_{n=1}^{1-1} a_{1,n}\ \vec{k}_n) \\
&= f(t^i\ h, \vec{u}(t^i)) = f(t^i\ h, (u_1(t^i), u_2(t^i))) \\
\vec{k}_2 &= f(t^i + c_2\ h, \vec{u}(t^i) + h \sum_{n=1}^{2-1} a_{2,n}\ \vec{k}_n) \\
&= f(t^i + \frac{1}{2}h, \vec{u}(t^i) + h\ a_{2,1}\ \vec{k}_1) \\
&= f(t^i + \frac{h}{2}, \vec{u}(t^i) + \frac{h}{2}\ \vec{k}_1) = f(t^i + \frac{h}{2}, (u_1(t^i) + \frac{h}{2}\ k_{1_1}, u_2(t^i) + \frac{h}{2}\ k_{1_2})) \\
\vec{u}(t^{i+1}) &= \vec{u}(t^i) + h\left(b_1\ \vec{k}_1 + b_2\ \vec{k}_2\right) \\
&= h\left(0 \times \vec{k}_1 + 1 \times \vec{k}_2\right) \\
&= h\ \vec{k}_2 = h(k_{2_1}, k_{2_2})
\end{aligned}
$$

Approximate ODE

Higher Order

Idea: reformulate the Cauchy problem with vectorial version

n order version of the Cauchy problem

$$\begin{cases} \partial_t^n u(t) & = & f(t, u(t), \partial_t^1 u(t), \ldots, \partial_t^{n-1} u(t)) \\ u(t_0) & = & u_0 \\ \partial_t^1 u(t^0) & = & u^{0,1} \\ \vdots & & \vdots \\ \partial_t^{n-1} u(t) & = & u^{0,n-1} \vdots \end{cases}$$

n order version, vectorial framework

$$\begin{cases} \partial_t^n \vec{v}(t) & = & F(t, u(t), \partial_t^1 u(t), \ldots, \partial_t^{n-1} u(t)) \\ \vec{v}(t^0) & = & (u(t^0), \partial_t^1 u_1(t^0), \ldots, \partial_t^{n-1} u_{n-1}(t^0)) \end{cases}$$

Approximate ODE

Higher Order – Example

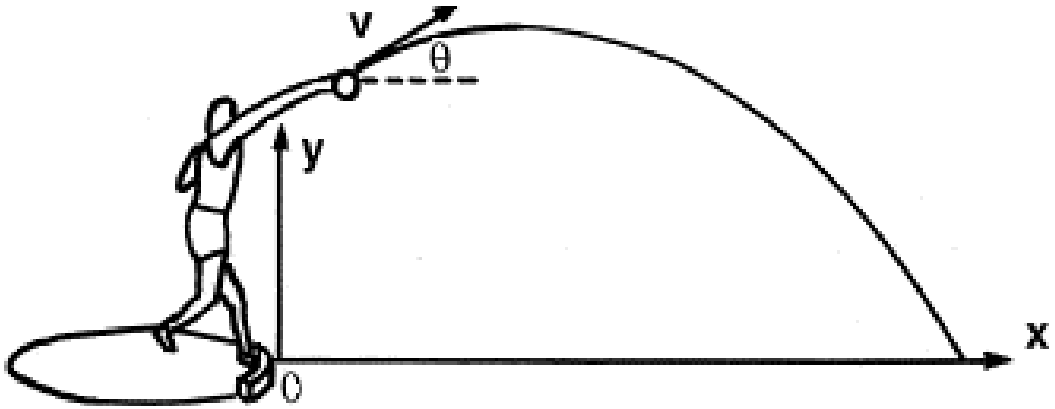
Shot put

A man (height: 1.80**m**) throws (translation)

shot put (a sphere $m = 7.26$**kg**, $\varnothing 13$**cm**) 20**cm** above his head with an initial speed of 12**m/s**

Approximate ODE

Higher Order – Example



$$\begin{cases} \partial_t^2 X(t) & = & - & V(t)[\alpha \partial_t^1 X(t) + \beta \partial_t^1 Y(t)] \\ \partial_t^2 Y(t) & = & & V(t)[\beta \partial_t^1 X(t) - \alpha \partial_t^1 Y(t)] - g \end{cases}$$

where

$\alpha = \rho C_x S \frac{1}{2m}$

$\beta = \rho C_z \frac{1}{2m}$

$g \approx 9.81\ \mathbf{m/s^2}$

$\theta = 45°$

ρ air density $= 1.29\mathbf{Kg/m^3}$

C_x friction coefficient $= 0.5$

C_z lift coefficient[a] $= 0.2$

S section $= 0.065^2\pi\ \mathbf{m^2}$

[a] Cœfficient de portance

Approximate ODE

Higher Order – Example

$$\alpha = \frac{1.29}{2}\frac{0.065^2\pi}{2\times 7.26} \approx 5.89\times 10^4 \qquad \beta = \frac{1.29}{5}\frac{0.065^2\pi}{2\times 7.26} = 2.36\times 10^4$$

$$\left\{\begin{array}{lcl} \partial_t^2 X(t) & = & -\quad V(t)\left[5.89\,\partial_t^1 X(t) + 2.36\,\partial_t^1 Y(t)\right]\times 10^4 \\ \partial_t^2 Y(t) & = & \quad\;\; V(t)\left[2.36\,\partial_t^1 X(t) - 5.89\,\partial_t^1 Y(t)\right]\times 10^4 - 9.81 \end{array}\right.$$

Approximate ODE

Higher Order – Example

$$\alpha = \frac{1.29}{2}\frac{0.065^2\pi}{2\times 7.26} \approx 5.89\times 10^4 \qquad \beta = \frac{1.29}{5}\frac{0.065^2\pi}{2\times 7.26} = 2.36\times 10^4$$

$$\left\{\begin{array}{lcl} \partial_t^2 X(t) & = & -\quad V(t)\left[5.89\,\partial_t^1 X(t) + 2.36\,\partial_t^1 Y(t)\right]\times 10^4 \\ \partial_t^2 Y(t) & = & \quad\;\; V(t)\left[2.36\,\partial_t^1 X(t) - 5.89\,\partial_t^1 Y(t)\right]\times 10^4 - 9.81 \end{array}\right.$$

Lets use $\hat{X}(t) = \partial_t X(t)$ and $\hat{X}(t) = \partial_t X(t)$:

$$\left\{\begin{array}{lcl} \partial_t \hat{X}(t) & = & -\quad V(t)\left[5.89\,\hat{X}(t) + 2.36\,\hat{Y}(t)\right]\times 10^4 \\ \partial_t \hat{Y}(t) & = & \quad\;\; V(t)\left[2.36\,\hat{X}(t) - 5.89\,\hat{Y}(t)\right]\times 10^4 - 9.81 \end{array}\right.$$

Now it's a first order EDO

Approximate ODE

Higher Order – Example

Cauchy problem becomes:

$$\begin{cases} \vec{u}(t) & = & f\left(t, (X(t), Y(t), \hat{X}(t), \hat{Y}(t))\right) \\ \vec{u}(t^0) & = & (0, 1.8 + 0.2, 12\cos(\theta = 45°), 12\sin(\theta = \frac{\pi}{4})) \end{cases}$$

RK2

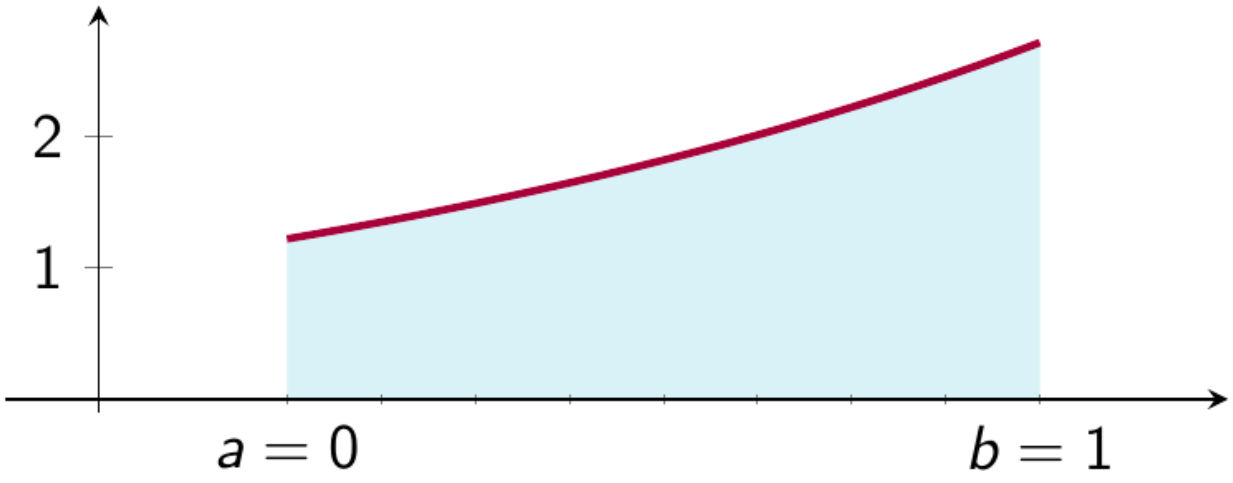
$$\begin{aligned}
\vec{k}_1 &= f(t^i\ h, (u_1(t^i), u_2(t^i), u_3(t^i), u_4(t^i))) \\
&= f(t^i\ h, (\mathbf{X}(t^i), \mathbf{Y}(t^i), \widehat{\mathbf{X}}(t^i), \widehat{\mathbf{Y}}(t^i))) \\
\vec{k}_2 &= f(t^i + \frac{h}{2}, (u_1(t^i) + \frac{h}{2}\ k_{1_1}, u_2(t^i) + \frac{h}{2}\ k_{1_2}, u_3(t^i) + \frac{h}{2}\ k_{1_3}, u_4(t^i) + \frac{h}{2}\ k_{1_4})) \\
&= f(t^i + \frac{h}{2}, (\mathbf{X}(t^i) + \frac{h}{2}\ k_{1_1}, \mathbf{Y}(t^i) + \frac{h}{2}\ k_{1_2}, \widehat{\mathbf{X}}(t^i) + \frac{h}{2}\ k_{1_3}, \widehat{\mathbf{Y}}(t^i) + \frac{h}{2}\ k_{1_4})) \\
\vec{u}(t^{i+1}) &= h(k_{2_1}, k_{2_2}, k_{2_3}, k_{2_4})
\end{aligned}$$

Content

1. Introduction
2. Approximate ODE
   - Principle
   - Euler method
   - Runge-Kutta methods
   - Example of RK2 First-Order
   - Vector-Valued Function
   - Higher Order
3. Numerical Integration
   - Principle
   - Lagrange Polynomial
   - Newton–Cotes rule
   - Newton–Cotes Formulas
4. Exercises
   - Hawk-Eye for Tennis
   - Integration

Numerical Integration

Compute $\int_0^1 e^x \, \mathrm{d}x$ **?**

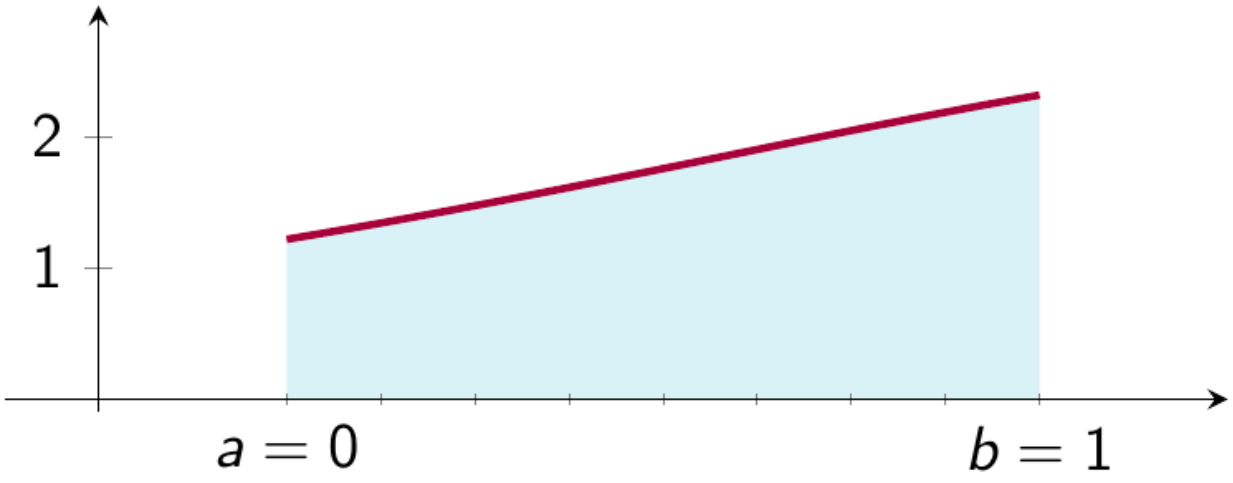


Easy game

$$\left[\int e^x \, \mathrm{d}x\right]_0^1 = [e^x]_0^1 = e^1 - e^0 = e - 1 \approx 1.7182$$

Numerical Integration

Compute $\int_0^1 e^{\sin(x)}\,\mathrm{d}x$?



More complex

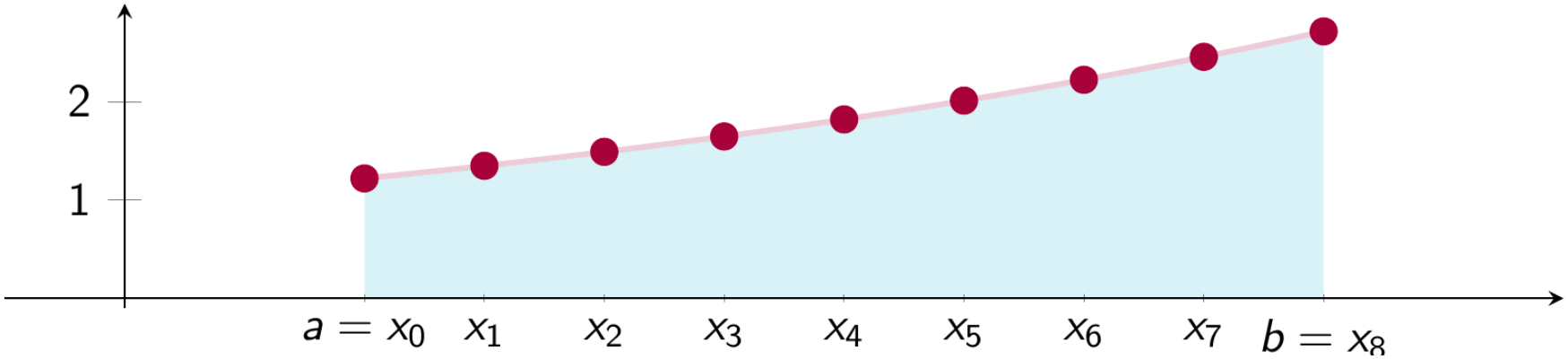
Numerical Integration

Principle

Numerical interpolation of $f(x)$ on $[a, b]$

Steps:

1. Evaluate $n+1$ data-points of the function $f(x)$ to integrate on $[a, b]$
2. Interpolate ($\approx$ approximation) the function $f(x)$ with a polynom $p(x)$ on the data points.
3. Integrate $p(x)$ on $[a, b]$



Data-points are **equaly spaced** ($x_i = a + ih$), $h = \frac{b-a}{n}$.
Note: Other methods exist for non-equaly spaced data-points.

Numerical Integration

Lagrange Polynomial

Interpolation

An interpolation of $f(x)$ is simpler function $\tilde{f}(x)$ that approximate and replace $f(x)$.

Polynomial Interpolation of degree n

A polynomial interpolation of $f(x)$ is a polynomial function $p(x)$ of degree n of the form:

$$p(x) = \sum_{i=0}^{n} \lambda_i x^i \qquad \text{where } \lambda \text{ are factors}$$

that approximate and replace $f(x)$.

Numerical Integration

Lagrange Polynomial

Lagrange Polynomial

Lagrange Polynomial is a model of Polynomial Interpolation $L(x)$ of degree n:

$$L(X) = \sum_{j=0}^{n} y_j \left(\prod_{i=0, i\neq j}^{n} \frac{X - x_i}{x_j - x_i} \right) = \sum_{i}^{n} y_i l_i(X)$$

where $l_i(X) = \prod_{j=0, j\neq i}^{n} \frac{X - x_j}{x_i - x_j} = \frac{X - x_0}{x_i - x_0} \dots \frac{X - x_{i-1}}{x_i - x_{i-1}} \frac{X - x_{i+1}}{x_i - x_{i+1}} \dots \frac{X - x_n}{x_i - x_n}$

$L(x)$ **exactly goes through** (x_i, y_i)

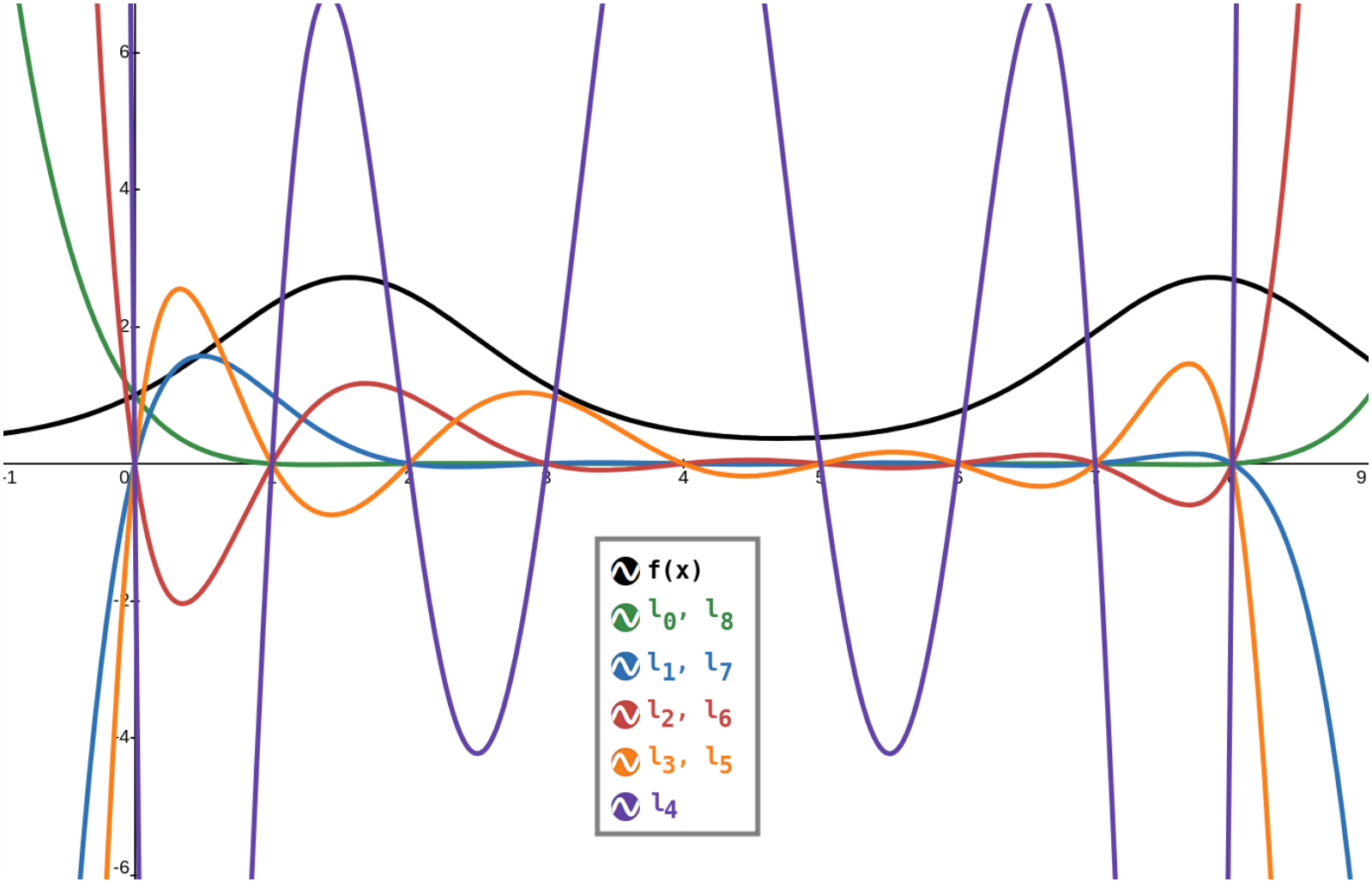
Hypothesis:
there are $n+1$ equaly spaced $n+1$ data-points of the form $(x_i, y_i) \in [a, b]$.

Note: Other methods exist for non-equaly spaced data-points.

Numerical Integration

Lagrange Polynomial

Kind of geometric construction (sum of l_i), e.g.: $f(x) = e^{\sin(x)}$



Numerical Integration

Lagrange Polynomial

Example of data-points of $f(x) = e^{\sin(x)}$) on $[\mathbf{a}, \mathbf{b}]$,
$$\mathbf{a} = \mathbf{0}, \mathbf{b} = \mathbf{8}, \mathbf{n} = \mathbf{8}, \mathbf{h} = \frac{\mathbf{8-0}}{\mathbf{n}} = \mathbf{1}$$

x	0	1	2	3	4	5	6	7	8
y	1	2.3198	2.4826	1.1516	0.4692	0.3833	0.7562	1.9289	2.6895

First step is to compute $l_i(X)$ where X are data-points:

$$l_i(X) = \prod_{j=0, j\neq i}^{n} \frac{X - x_j}{x_i - x_j} = \frac{X - x_0}{x_i - x_0} \cdots \frac{X - x_{i-1}}{x_i - x_{i-1}} \frac{X - x_{i+1}}{x_i - x_{i+1}} \cdots \frac{X - x_n}{x_i - x_n}$$

Numerical Integration

Lagrange Polynomial

$$
\begin{aligned}
l_0 &= \frac{x-x_1}{x_0-x_1}\,\frac{x-x_2}{x_0-x_2}\,\frac{x-x_3}{x_0-x_3}\,\frac{x-x_4}{x_0-x_4}\,\frac{x-x_5}{x_0-x_5}\,\frac{x-x_6}{x_0-x_6}\,\frac{x-x_7}{x_0-x_7}\,\frac{x-x_8}{x_0-x_8} \\
&= \frac{x-1}{0-1}\,\frac{x-2}{0-2}\,\frac{x-3}{0-3}\,\frac{x-4}{0-4}\,\frac{x-5}{0-5}\,\frac{x-6}{0-6}\,\frac{x-7}{0-7}\,\frac{x-8}{0-8} \\
&= \frac{(x-1)(x-2)(x-3)(x-4)(x-5)(x-6)(x-7)(x-8)}{(-1)(-2)(-3)(-4)(-5)(-6)(-7)(-8)} \\
&= \frac{40320-109584x+118124x^2-67284x^3+22449x^4}{8!} \\
&\quad + \frac{-4536x^5+546x^6-36x^7+x^8}{8!} \\
&= 1-\frac{761x}{280}+\frac{29531x^2}{10080}-\frac{267x^3}{160}+\frac{1069x^4}{1920}-\frac{9x^5}{80}+\frac{13x^6}{960}-\frac{x^7}{1120}+\frac{x^8}{40320}
\end{aligned}
$$

Numerical Integration

Lagrange Polynomial

$$
\begin{aligned}
l_0 &= 1 - \frac{761x}{280} + \frac{29531x^2}{10080} - \frac{267x^3}{160} + \frac{1069x^4}{1920} - \frac{9x^5}{80} + \frac{13x^6}{960} - \frac{x^7}{1120} + \frac{x^8}{40320} \\
l_1 &= 8x - \frac{481x^2}{35} + \frac{349x^3}{36} - \frac{329x^4}{90} + \frac{115x^5}{144} - \frac{73x^6}{720} + \frac{x^7}{144} - \frac{x^8}{5040} \\
l_2 &= -14x + \frac{621x^2}{20} - \frac{18353x^3}{720} + \frac{15289x^4}{1440} - \frac{179x^5}{72} + \frac{239x^6}{720} - \frac{17x^7}{720} + \frac{x^8}{1440} \\
l_3 &= \frac{56x}{3} - \frac{2003x^2}{45} + \frac{797x^3}{20} - \frac{268x^4}{15} + \frac{71x^5}{16} - \frac{149x^6}{240} + \frac{11x^7}{240} - \frac{x^8}{720} \\
l_4 &= -210x + \frac{2073x^2}{4} - \frac{1457x^3}{3} + \frac{10993x^4}{48} - \frac{179x^5}{3} + \frac{209x^6}{24} - \frac{2x^7}{3} + \frac{x^8}{48} \\
l_5 &\Leftrightarrow= l_3 \\
l_6 &= l_2 \\
l_7 &= l_1 \\
l_8 &= l_0
\end{aligned}
$$

Numerical Integration

Lagrange Polynomial

Second step is to compute $L(X)$:

$$L(X) = \sum_{i}^{n} y_i l_i(X)$$

$$\begin{aligned} L(x) &= y_0 l_0(x) + y_1(x) + \ldots + y_8 l_8(x) \\ &= 1 \times l_0(x) + 2.3198 \times l_1(x) + \ldots \end{aligned}$$

Numerical Integration

Newton–Cotes rule

Want to integrate $f(x)$ on $[a, b]$, $L(x)$ is Lagrange polynomial

$$\int_a^b f(x)\,\mathrm{d}x \approx \int_a^b L(x)\,\mathrm{d}x$$

$$\begin{aligned}
\int_a^b L(x)\,\mathrm{d}x &= \int_a^b \sum_i^n y_i l_i(x)\,\mathrm{d}x \\
&= \sum_i^n \int_a^b y_i l_i(x)\,\mathrm{d}x \\
&= \sum_i^n y_i \int_a^b l_i(x)\,\mathrm{d}x \\
&= \frac{b-a}{b-a} \sum_i^n y_i \int_a^b l_i(x)\,\mathrm{d}x \\
&= (b-a) \sum_i^n y_i \underbrace{\frac{\int_a^b l_i(x)\,\mathrm{d}x}{b-a}}_{\text{Newton-Cotes uses symbol } \lambda_i}
\end{aligned}$$

Newton-Cotes formulas to integrate $I(f(x))$ the function $f(x)$ look like $\mathbf{I(f(x)) = (b-a)\sum_{j=0}^{n} \lambda_j y_j}$

Numerical Integration

Newton–Cotes rule

Instead of a **lot of computation** and **numerical errors**
$\Rightarrow$ Find real values for λ_i with a system of linear equations

To simplify mathematical expression, assume that $f(x)$ looks

$$f(x) = (x - a)^j \qquad , j \in [0, n]$$

Hypothesis: $f(x)$ is a **polynomial function**. Not always the case but $f(x)$ **can be interpolated** on $[a, b]$ with a *small* error $\Rightarrow$ can **assume polynomial function**.

Numerical Integration

Newton–Cotes rule

$$
\begin{aligned}
\int_a^b f(x)\,\mathrm{d}x &= I(f(x)) \\
\left[\frac{(x-a)^{j+1}}{j+1}\right]_a^b &= (b-a)\left(\sum_{i=0}^{n}\lambda_i y_i\right) \\
\frac{(b-a)^{j+1}}{j+1} - \frac{(a-a)^{j+1}}{j+1} &= (b-a)\left(\sum_{i=0}^{n}\lambda_i f(y_i)\right) \\
\frac{(\mathbf{b}-\mathbf{a})^{\mathbf{j}+\mathbf{1}}}{j+1} &= (b-a)\left(\sum_{i=0}^{n}\lambda_i (x-a)^j\right) \\
&= (\mathbf{b}-\mathbf{a})^{\mathbf{j}+\mathbf{1}}\left(\sum_{i=0}^{n}\lambda_i \frac{(x-a)^j}{(b-a)^j}\right) \\
\frac{1}{j+1} &= \sum_{i=0}^{n}\lambda_i \underbrace{\frac{(x-a)^j}{(b-a)^j}}_{\text{Write } t_i \text{ for } (x-a)+(b-a)}
\end{aligned}
$$

Numerical Integration

Newton–Cotes rule

Example $\int_0^4 f(x) = e^x \, \mathrm{d}x$

$$t_i = \frac{x-a}{b-a} = \frac{x-0}{4-0} = \frac{1}{4}x$$

x	0	1	2	3	4
y	1	2.718	7.389	20.085	54.598
t_i	0	0.25	0.5	0.75	1

System of equations $\left(\lambda_j = \frac{1}{j+1}\right)$ is

$$\begin{aligned}
\lambda_0 \times 0^0 + \lambda_1 \times 0.25^0 + \lambda_2 \times 0.5^0 + \lambda_3 \times 0.75^0 + \lambda_4 \times 1^0 &= 1 \\
\lambda_0 \times 0^1 + \lambda_1 \times 0.25^1 + \lambda_2 \times 0.5^1 + \lambda_3 \times 0.75^1 + \lambda_4 \times 1^1 &= 1/2 \\
\lambda_0 \times 0^2 + \lambda_1 \times 0.25^2 + \lambda_2 \times 0.5^2 + \lambda_3 \times 0.75^2 + \lambda_4 \times 1^2 &= 1/3 \\
\lambda_0 \times 0^3 + \lambda_1 \times 0.25^3 + \lambda_2 \times 0.5^3 + \lambda_3 \times 0.75^3 + \lambda_4 \times 1^3 &= 1/4 \\
\lambda_0 \times 0^4 + \lambda_1 \times 0.25^4 + \lambda_2 \times 0.5^4 + \lambda_3 \times 0.75^4 + \lambda_4 \times 1^4 &= 1/5
\end{aligned}$$

Numerical Integration

Newton–Cotes rule

Example $\int_0^4 f(x) = e^x \, \mathrm{d}x$

$$t_i = \frac{x-a}{b-a} = \frac{x-0}{4-0} = \frac{1}{4}x$$

x	0	1	2	3	4
y	1	2.718	7.389	20.085	54.598
t_i	0	0.25	0.5	0.75	1

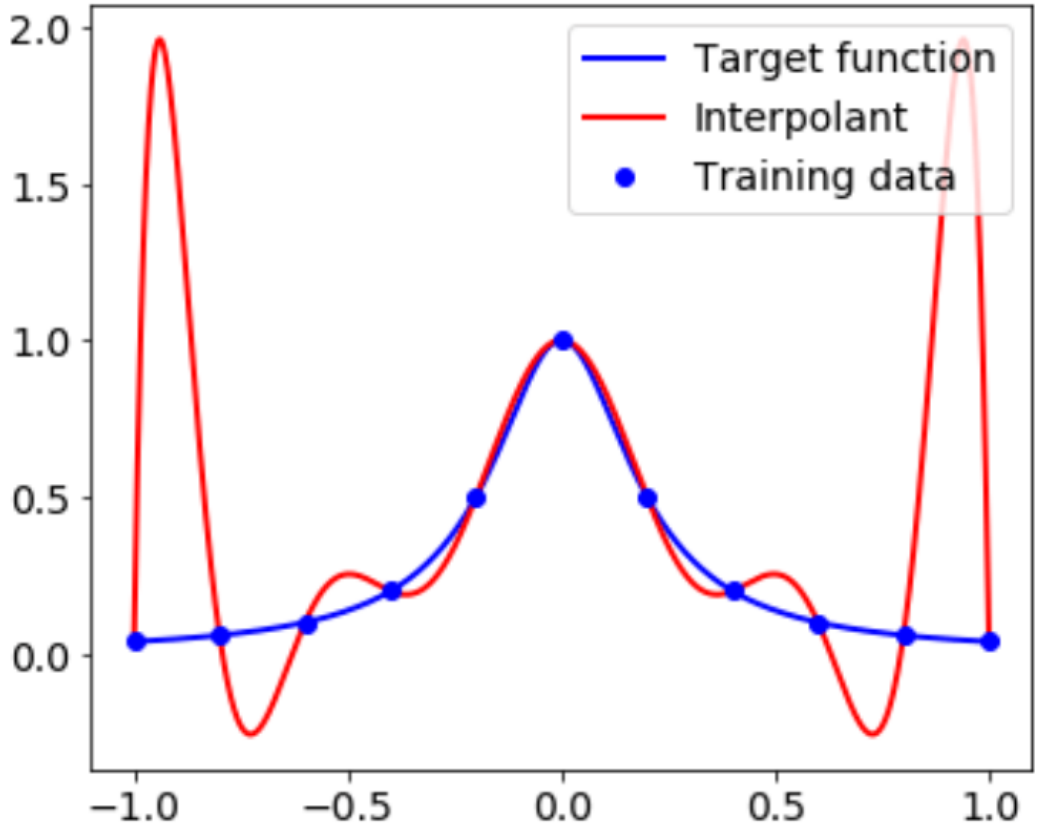
$$\lambda_0 = \frac{7}{90} \qquad \lambda_1 = \frac{32}{90} \qquad \lambda_2 = \frac{12}{90} \qquad \lambda_3 = \frac{32}{90} \qquad \lambda_4 = \frac{7}{90}$$

$$\begin{aligned} \mathbf{I(f(x))} &= (4-0)\left(\frac{7}{90} \times 1\frac{32}{90} \times 2.718\frac{12}{90} \times 7.389\frac{32}{90} \times 20.085\frac{7}{90} \times 54.598\right) \\ &\approx 53.359 \end{aligned}$$

Numerical Integration

Newton–Cotes Formulas

Runge's phenomenon: *more points, more errors*

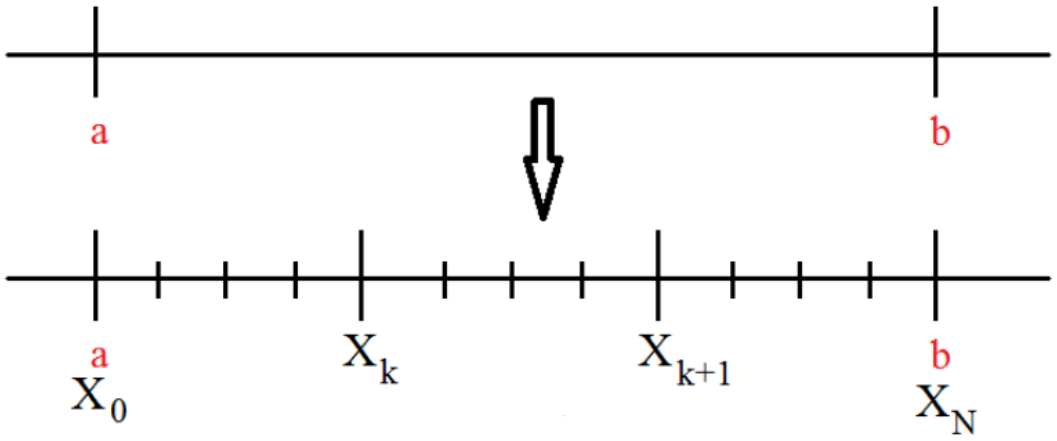


Either few data-points (lower n, bad precision) or few-points formulas

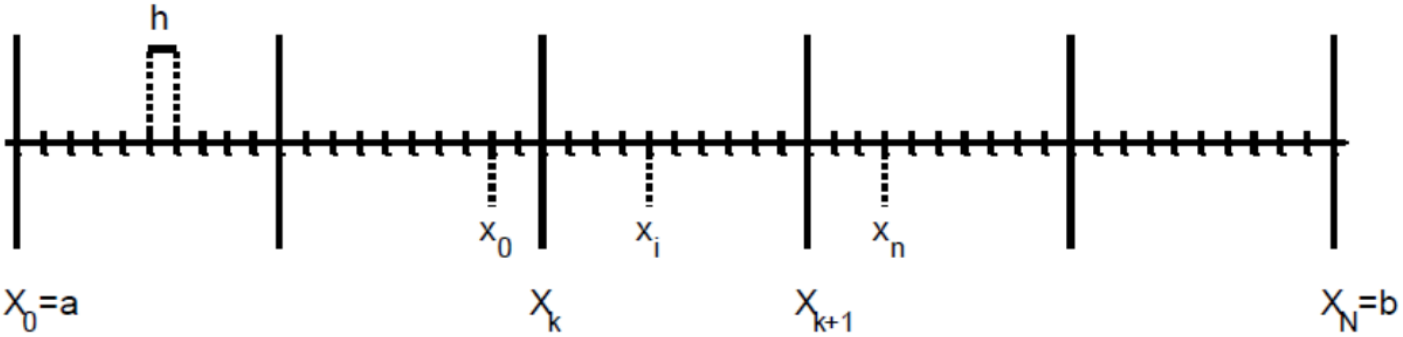
Numerical Integration

Newton–Cotes Formulas

Idea: **subdivide to evalutes few points with more** λ_i



N intervals on [a,b] delimited by $[X_k, X_{k+1}]$, $H = \frac{b-a}{N}$



X_k, X_{k+1} is subdivided in m intervals, $h = \frac{H}{m}$

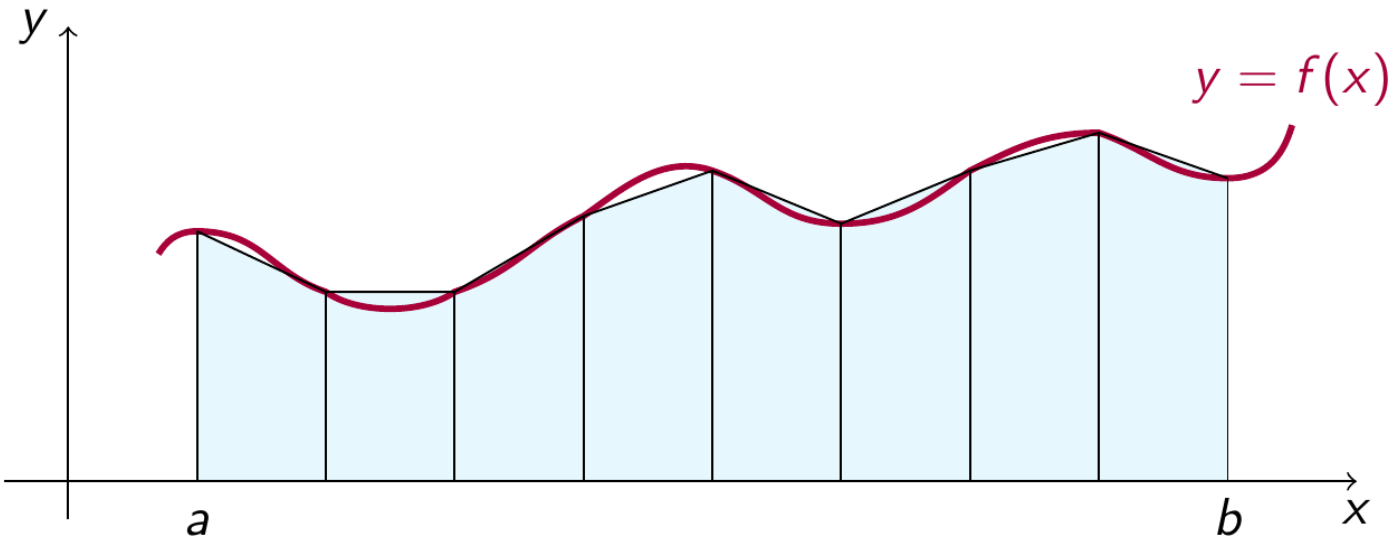
Numerical Integration

Newton–Cotes Formulas

On each interval construct a polynom (i.e. solve a system)
The whole system is to solve:

$$\frac{m^{j+1}}{j+1} = \sum_{0}^{n} m\lambda_i (r+i)^j \qquad , j \in [0, n]$$

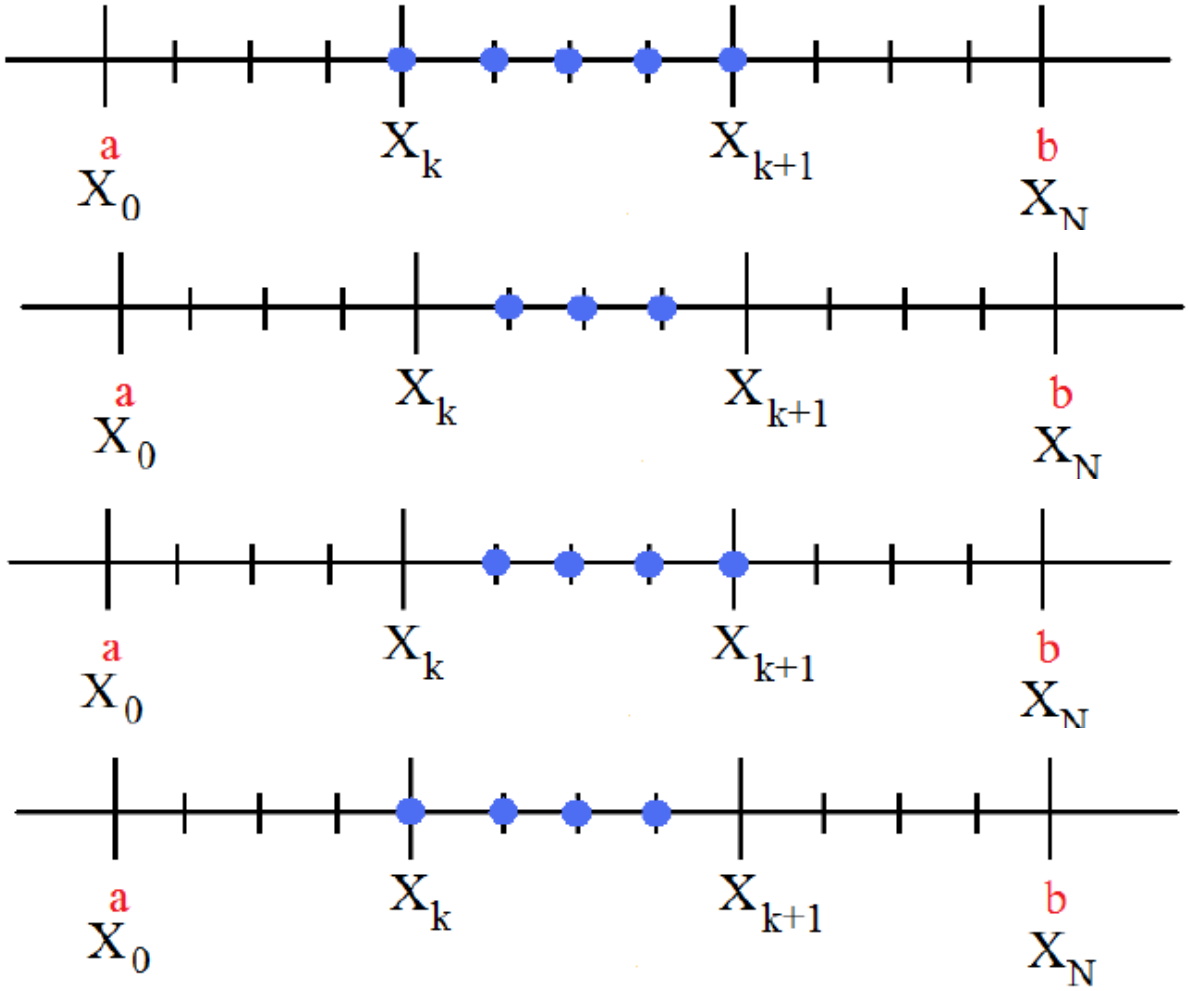
Example: trapzeoidal rule



Numerical Integration

Newton–Cotes Formulas

Closed formulas
$(n = m, r = 0)$



Open formulas
$(n = m - 2, r = 1)$

Left-open formulas
$(n = m - 1, r = 1)$

Right-open formulas
$(n = m - 1, r = 0)$

Numerical Integration

Closed Newton–Cotes Formulas

Trapezoidal rule

$$h = b - a \qquad \frac{1}{2}h\left(f(x_0) + f(x_1)\right) \qquad \varepsilon = -\frac{1}{12}h^3\ f(x_{\max})$$

Simpson's rule

$$h = \frac{b-a}{2} \qquad \frac{1}{3}h\left(f(x_0) + 4f(x_1) + f(x_2)\right) \qquad \varepsilon = -\frac{1}{90}h^5\ f(x_{\max})$$

Simpson's 3/8 rule

$$h = \frac{b-a}{3} \qquad \frac{3}{8}h\left(f(x_0) + 3f(x_1) + 3f(x_3) + f_3\right) \qquad \varepsilon = -\frac{3}{80}h^5\ f(x_{\max})$$

Boole's rule

$$\frac{2}{45}h\big(7f(x_0) + 32f(x_1) + 12f(x_2) + 32f(x_3) + 7f(x_4)\big)$$

$$h = \frac{b-a}{4} \qquad \varepsilon = -\frac{8}{945}h^7\ f(x_{\max})$$

Numerical Integration

Open Newton–Cotes Formulas

Midpoint/Rectangle rule

$$h = \frac{b-a}{2} \qquad 2hf(x_0) \qquad \varepsilon = \frac{1}{3}h^3\ f(x_{\max})$$

Milne's rule

$$h = \frac{b-a}{4} \qquad \frac{4}{3}h\left(2f(x_0) - f(x_1) + 2f(x_2)\right) \qquad \varepsilon = \frac{14}{45}h^5\ f(x_{\max})$$

It exists other rules

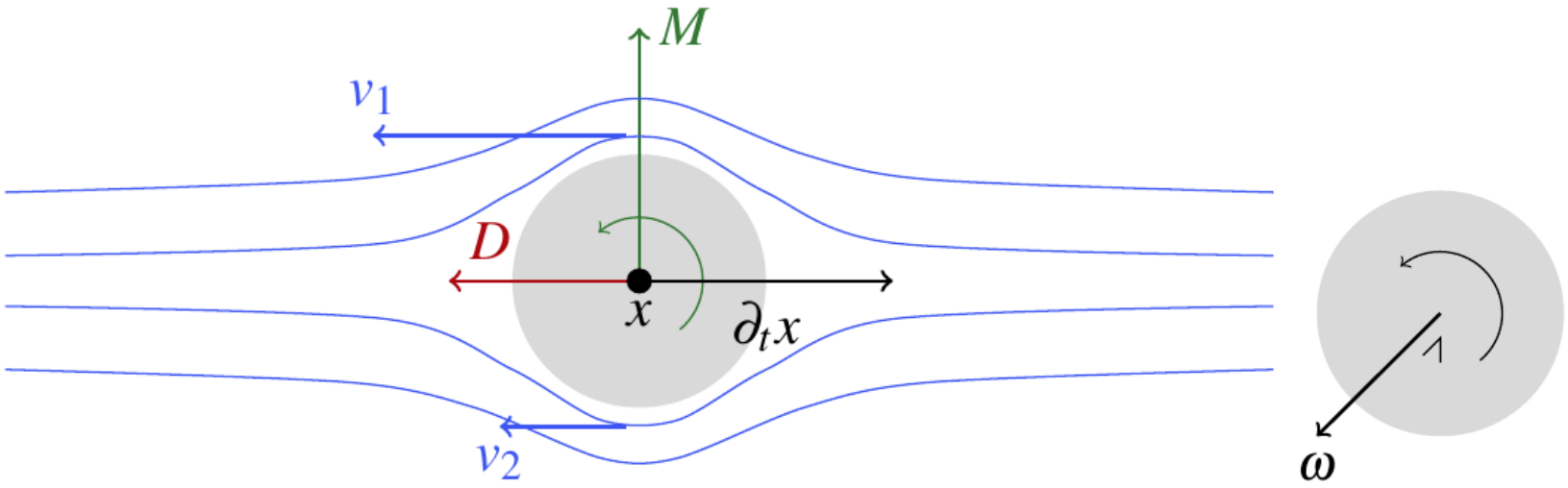
Content

1. Introduction
2. Approximate ODE
   - Principle
   - Euler method
   - Runge-Kutta methods
   - Example of RK2 First-Order
   - Vector-Valued Function
   - Higher Order
3. Numerical Integration
   - Principle
   - Lagrange Polynomial
   - Newton–Cotes rule
   - Newton–Cotes Formulas
4. Exercises
   - Hawk-Eye for Tennis
   - Integration

Exercises

Hawk-Eye for Tennis

Magnus Effect



v_1 and v_2 are apparent air speeds

ω vector of rotation ($\approx$ constant, right-hand rule)

ρ $= 1.23\text{kg m}^{-3}$ sea-level air density

R $\approx 3.35\text{cm}$ ball radius

m $= 0.58\text{g}$ is the ball mass

$|\bullet|$ is the Euclidean norm

$\times$ is the vector product

Exercises

Hawk-Eye for Tennis

Ball motion vector

$$M = \frac{1}{2}\rho C_M \pi R^2 \left|\partial_t x\right|^2 \frac{\omega}{|\omega|} \times \frac{\partial_t x}{|\partial_t x|}$$

Air resistance vector

$$D = -\frac{1}{2}\rho C_D \pi R^2 \left|\partial_t x\right|^2 \frac{\partial_t x}{|\partial_t x|}$$

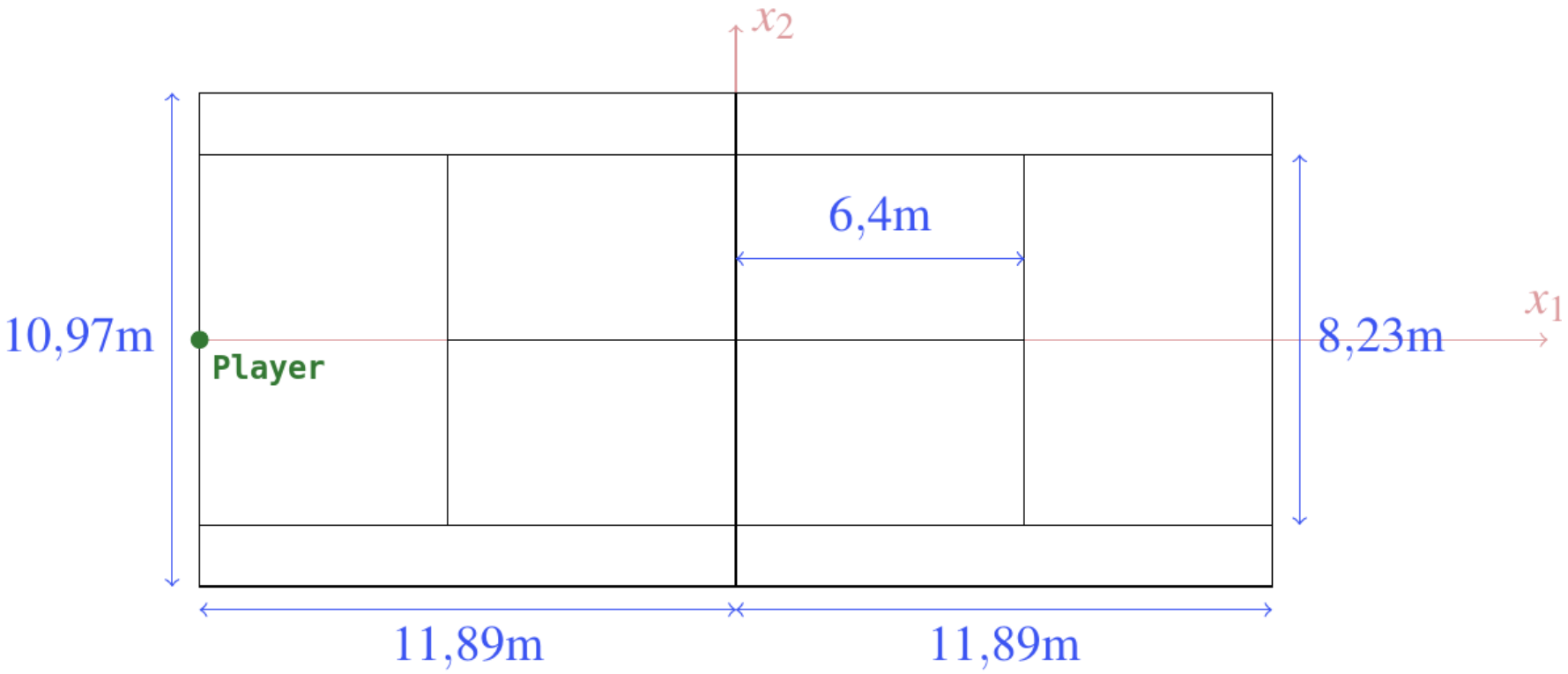
$$C_M = \frac{1}{2.022 + 0.981 \cdot q} \quad C_D = 0.508 + \left(\frac{1}{22.053 + 4.196 \cdot q^{\frac{5}{2}}}\right)^{\frac{2}{5}} \quad q = \frac{|\partial_t x|}{R|\omega|}$$

Exercises

Hawk-Eye for Tennis

From Newton's laws of motion, the **movement** of the **center** x of the **ball** is

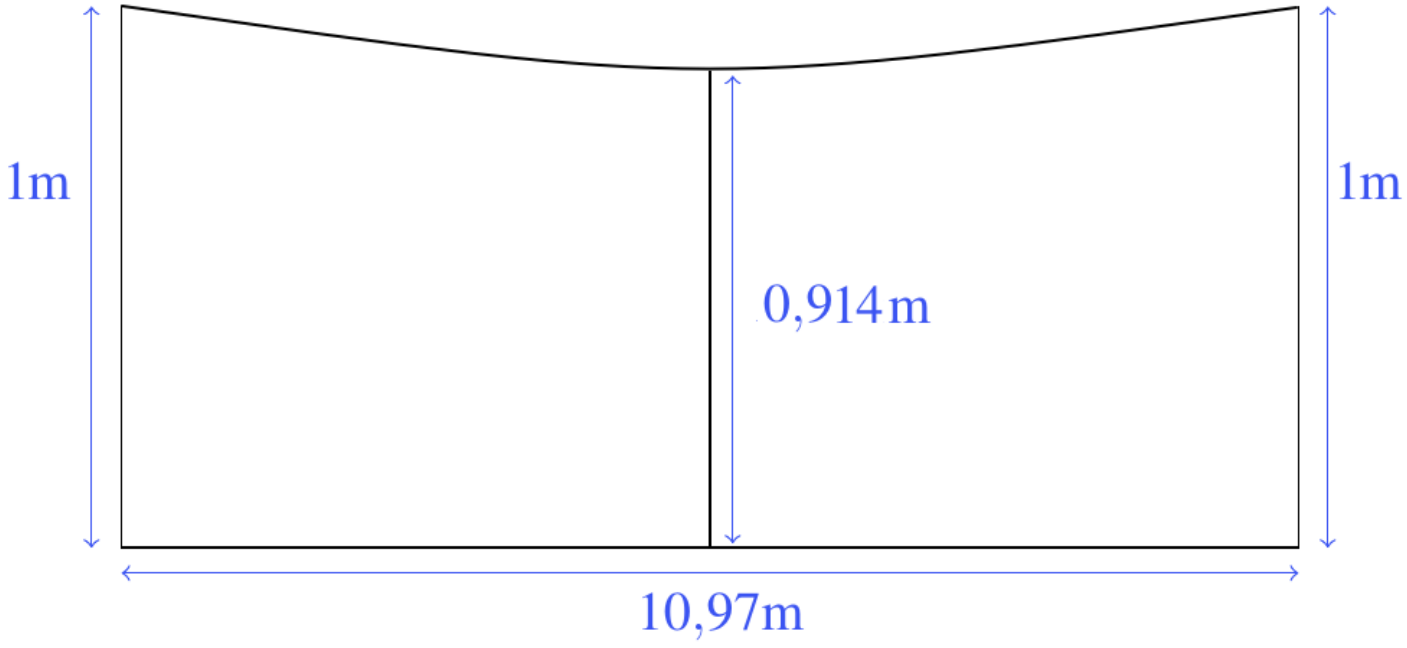
$$m\partial_t^2 x(t) = (0, 0, -mg) + D + M$$



Exercises

Hawk-Eye for Tennis

Net has a catenary curve shape



$$f(x) \equiv y = a\cosh\left(\frac{x}{a}\right) \Leftrightarrow y = \frac{a}{2}\left(e^{\frac{x}{a}} + e^{-\frac{x}{a}}\right)$$

Exercises

Hawk-Eye for Tennis

3 categories of shots:

- **Flat** hit without spin: $\omega = (0, 0, 0)$
- **Topspin** hit by brushing up the back of the ball: $\omega_i > 0$
- **Slice** hit by brushing down the back of the ball: $\omega_i < 0$

where i is the coordinate (right-hand) corresponding to x_3.

Flat shot implies $\omega = (0, 0, 0) \Rightarrow q \to \infty \Rightarrow C_M = 0, C_D = 0.508$

Exercises

Hawk-Eye for Tennis

Assume that positions of the ball are of the form: $x(t) = (x_1(t), x_2(t), x_3(t))$ where

x_1 is the coordinate between the player and the net.

x_2 is the coordinate between the two sides of the court.

x_3 is the coordinate between the ground and the sky.

Assume three shots: flat, topspin ($|\omega| = 30$) and, slice ($|\omega| = 30$).

With an initial angle of 10° between x_1 and x_3 at the hit level:

1. What are the initial conditions (flat, topspin, slice)?
2. Define the Cauchy problem.
3. Implement RK4.
4. What is the minimal (resp. maximal) speed to be on the court?

After that, you can try `DifferentialEquations.jl`

Exercises

Integration

No fun idea ... :-(

Just compute

$$\int_1^4 \cos((ln)(x))\,\mathrm{d}x$$

with Simpson's rule but avoid Runge phenomenon